JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZipOutputStream

java.lang.Object
  java.io.OutputStream
    java.io.FilterOutputStream
      java.util.zip.DeflaterOutputStream
        java.util.zip.ZipOutputStream

All Implemented Interfaces:

`Closeable`, `Flushable`, `AutoCloseable`

Direct Known Subclasses:

`JarOutputStream`

```
public class ZipOutputStream
extends DeflaterOutputStream
```

This class implements an output stream filter for writing files in the ZIP file format. Includes support for both compressed and uncompressed entries.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

By default, the UTF-8 charset is used to encode entry names and comments. `ZipOutputStream(OutputStream, Charset)` may be be used to specify an alternative charset.

Since:

1.1

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**`CENATT`**	Central directory (CEN) header internal file attributes field offset.
`static final int`	**`CENATX`**	Central directory (CEN) header external file attributes field offset.
`static final int`	**`CENCOM`**	Central directory (CEN) header comment length field offset.
`static final int`	**`CENCRC`**	

		Central directory (CEN) header uncompressed file crc-32 value field offset.
static final int	**CENDSK**	Central directory (CEN) header disk number start field offset.
static final int	**CENEXT**	Central directory (CEN) header extra field length field offset.
static final int	**CENFLG**	Central directory (CEN) header encrypt, decrypt flags field offset.
static final int	**CENHDR**	Central directory (CEN) header size in bytes (including signature).
static final int	**CENHOW**	Central directory (CEN) header compression method field offset.
static final int	**CENLEN**	Central directory (CEN) header uncompressed size field offset.
static final int	**CENNAM**	Central directory (CEN) header filename length field offset.
static final int	**CENOFF**	Central directory (CEN) header LOC header offset field offset.
static final long	**CENSIG**	Central directory (CEN) header signature.
static final int	**CENSIZ**	Central directory (CEN) header compressed size field offset.
static final int	**CENTIM**	Central directory (CEN) header modification time field offset.

static final int	**CENVEM**	Central directory (CEN) header version made by field offset.
static final int	**CENVER**	Central directory (CEN) header version needed to extract field offset.
static final int	**DEFLATED**	Compression method for compressed (DEFLATED) entries.
static final int	**ENDCOM**	End of central directory (END) header ZIP file comment length field offset.
static final int	**ENDHDR**	End of central directory (END) header size in bytes (including signature).
static final int	**ENDOFF**	End of central directory (END) header offset for the first CEN header field offset.
static final long	**ENDSIG**	End of central directory (END) header signature.
static final int	**ENDSIZ**	End of central directory (END) header central directory size in bytes field offset.
static final int	**ENDSUB**	End of central directory (END) header number of entries on this disk field offset.
static final int	**ENDTOT**	End of central directory (END) header total number of entries field offset.
static final int	**EXTCRC**	

		Extra local (EXT) header uncompressed file crc-32 value field offset.
static final int	**EXTHDR**	Extra local (EXT) header size in bytes (including signature).
static final int	**EXTLEN**	Extra local (EXT) header uncompressed size field offset.
static final long	**EXTSIG**	Extra local (EXT) header signature.
static final int	**EXTSIZ**	Extra local (EXT) header compressed size field offset.
static final int	**LOCCRC**	Local file (LOC) header uncompressed file crc-32 value field offset.
static final int	**LOCEXT**	Local file (LOC) header extra field length field offset.
static final int	**LOCFLG**	Local file (LOC) header general purpose bit flag field offset.
static final int	**LOCHDR**	Local file (LOC) header size in bytes (including signature).
static final int	**LOCHOW**	Local file (LOC) header compression method field offset.
static final int	**LOCLEN**	Local file (LOC) header uncompressed size field offset.
static final int	**LOCNAM**	Local file (LOC) header filename length field offset.

static final long	**LOCSIG**	Local file (LOC) header signature.
static final int	**LOCSIZ**	Local file (LOC) header compressed size field offset.
static final int	**LOCTIM**	Local file (LOC) header modification time field offset.
static final int	**LOCVER**	Local file (LOC) header version needed to extract field offset.
static final int	**STORED**	Compression method for uncompressed (STORED) entries.

Fields declared in class DeflaterOutputStream

`buf`, `def`

Fields declared in class FilterOutputStream

`out`

Constructor Summary

Constructors

Constructor	Description
ZipOutputStream(OutputStream out)	Creates a new ZIP output stream using the UTF-8 charset to encode entry names and comments.
ZipOutputStream(OutputStream out, Charset charset)	Creates a new ZIP output stream using the specified charset to encode entry names and comments.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	close()	Closes the underlying stream and the stream being filtered after the contents of the ZIP output stream are fully written.
void	closeEntry()	Closes the current ZIP entry and positions the stream for writing the next entry.
void	finish()	Finishes writing the contents of the ZIP output stream without closing the underlying stream.
void	putNextEntry (ZipEntry e)	Begins writing a new ZIP file entry and positions the stream to the start of the entry data.
void	setComment (String comment)	Sets the ZIP file comment.
void	setLevel(int level)	Sets the compression level for subsequent entries which are DEFLATED.
void	setMethod(int method)	Sets the default compression method for subsequent entries.
void	write(byte[] b, int off, int len)	Writes an array of bytes to the current ZIP entry data.

Methods declared in class DeflaterOutputStream

deflate, flush, write

Methods declared in class FilterOutputStream

`write`

Methods declared in class OutputStream

`nullOutputStream`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

STORED

`public static final int STORED`

Compression method for uncompressed (STORED) entries.

See Also:

Constant Field Values

DEFLATED

`public static final int DEFLATED`

Compression method for compressed (DEFLATED) entries.

See Also:

Constant Field Values

LOCSIG

`static final long LOCSIG`

Local file (LOC) header signature.

See Also:

Constant Field Values

EXTSIG

`static final long EXTSIG`

Extra local (EXT) header signature.

See Also:

Constant Field Values

CENSIG

`static final long CENSIG`

Central directory (CEN) header signature.

See Also:

Constant Field Values

ENDSIG

`static final long ENDSIG`

End of central directory (END) header signature.

See Also:

Constant Field Values

LOCHDR

`static final int LOCHDR`

Local file (LOC) header size in bytes (including signature).

See Also:

Constant Field Values

EXTHDR

`static final int EXTHDR`

Extra local (EXT) header size in bytes (including signature).

See Also:

Constant Field Values

CENHDR

`static final int CENHDR`

Central directory (CEN) header size in bytes (including signature).

See Also:

Constant Field Values

ENDHDR

```
static final int ENDHDR
```

End of central directory (END) header size in bytes (including signature).

See Also:

Constant Field Values

LOCVER

```
static final int LOCVER
```

Local file (LOC) header version needed to extract field offset.

See Also:

Constant Field Values

LOCFLG

```
static final int LOCFLG
```

Local file (LOC) header general purpose bit flag field offset.

See Also:

Constant Field Values

LOCHOW

```
static final int LOCHOW
```

Local file (LOC) header compression method field offset.

See Also:

Constant Field Values

LOCTIM

```
static final int LOCTIM
```

Local file (LOC) header modification time field offset.

See Also:

Constant Field Values

LOCCRC

```
static final int LOCCRC
```

Local file (LOC) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

LOCSIZ

`static final int LOCSIZ`

Local file (LOC) header compressed size field offset.

See Also:

Constant Field Values

LOCLEN

`static final int LOCLEN`

Local file (LOC) header uncompressed size field offset.

See Also:

Constant Field Values

LOCNAM

`static final int LOCNAM`

Local file (LOC) header filename length field offset.

See Also:

Constant Field Values

LOCEXT

`static final int LOCEXT`

Local file (LOC) header extra field length field offset.

See Also:

Constant Field Values

EXTCRC

`static final int EXTCRC`

Extra local (EXT) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

EXTSIZ

```
static final int EXTSIZ
```

Extra local (EXT) header compressed size field offset.

See Also:

Constant Field Values

EXTLEN

```
static final int EXTLEN
```

Extra local (EXT) header uncompressed size field offset.

See Also:

Constant Field Values

CENVEM

```
static final int CENVEM
```

Central directory (CEN) header version made by field offset.

See Also:

Constant Field Values

CENVER

```
static final int CENVER
```

Central directory (CEN) header version needed to extract field offset.

See Also:

Constant Field Values

CENFLG

```
static final int CENFLG
```

Central directory (CEN) header encrypt, decrypt flags field offset.

See Also:

Constant Field Values

CENHOW

```
static final int CENHOW
```

Central directory (CEN) header compression method field offset.

See Also:

Constant Field Values

CENTIM

`static final int CENTIM`

Central directory (CEN) header modification time field offset.

See Also:

Constant Field Values

CENCRC

`static final int CENCRC`

Central directory (CEN) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

CENSIZ

`static final int CENSIZ`

Central directory (CEN) header compressed size field offset.

See Also:

Constant Field Values

CENLEN

`static final int CENLEN`

Central directory (CEN) header uncompressed size field offset.

See Also:

Constant Field Values

CENNAM

`static final int CENNAM`

Central directory (CEN) header filename length field offset.

See Also:

Constant Field Values

CENEXT

```
static final int CENEXT
```

Central directory (CEN) header extra field length field offset.

See Also:

Constant Field Values

CENCOM

```
static final int CENCOM
```

Central directory (CEN) header comment length field offset.

See Also:

Constant Field Values

CENDSK

```
static final int CENDSK
```

Central directory (CEN) header disk number start field offset.

See Also:

Constant Field Values

CENATT

```
static final int CENATT
```

Central directory (CEN) header internal file attributes field offset.

See Also:

Constant Field Values

CENATX

```
static final int CENATX
```

Central directory (CEN) header external file attributes field offset.

See Also:

Constant Field Values

CENOFF

```
static final int CENOFF
```

Central directory (CEN) header LOC header offset field offset.

See Also:

Constant Field Values

ENDSUB

`static final int ENDSUB`

End of central directory (END) header number of entries on this disk field offset.

See Also:

Constant Field Values

ENDTOT

`static final int ENDTOT`

End of central directory (END) header total number of entries field offset.

See Also:

Constant Field Values

ENDSIZ

`static final int ENDSIZ`

End of central directory (END) header central directory size in bytes field offset.

See Also:

Constant Field Values

ENDOFF

`static final int ENDOFF`

End of central directory (END) header offset for the first CEN header field offset.

See Also:

Constant Field Values

ENDCOM

`static final int ENDCOM`

End of central directory (END) header ZIP file comment length field offset.

See Also:

Constant Field Values

Constructor Details

ZipOutputStream

```
public ZipOutputStream(OutputStream out)
```

Creates a new ZIP output stream using the UTF-8 `charset` to encode entry names and comments.

Parameters:

`out` - the actual output stream

ZipOutputStream

```
public ZipOutputStream(OutputStream out,
                       Charset charset)
```

Creates a new ZIP output stream using the specified `charset` to encode entry names and comments.

Parameters:

`out` - the actual output stream

`charset` - the charset to be used to encode entry names and comments

Since:

1.7

Method Details

setComment

```
public void setComment(String comment)
```

Sets the ZIP file comment. If `comment` is an empty string or `null` then the output will have no ZIP file comment.

Parameters:

`comment` - the comment string, or an empty string or null for no comment

Throws:

`IllegalArgumentException` - if the length of the specified ZIP file comment is greater than 0xFFFF bytes or if the `comment` contains characters that cannot be mapped by the `Charset` used to encode entry names and comments

setMethod

```
public void setMethod(int method)
```

Sets the default compression method for subsequent entries. This default will be used whenever the compression method is not specified for an individual ZIP file entry, and is initially set to DEFLATED.

Parameters:

`method` - the default compression method

Throws:

`IllegalArgumentException` - if the specified compression method is invalid

setLevel

```
public void setLevel(int level)
```

Sets the compression level for subsequent entries which are DEFLATED. The default setting is DEFAULT_COMPRESSION.

Parameters:

`level` - the compression level (0-9)

Throws:

`IllegalArgumentException` - if the compression level is invalid

putNextEntry

```
public void putNextEntry(ZipEntry e)
                  throws IOException
```

Begins writing a new ZIP file entry and positions the stream to the start of the entry data. Closes the current entry if still active.

The default compression method will be used if no compression method was specified for the entry. When writing a compressed (DEFLATED) entry, and the compressed size has not been explicitly set with the `ZipEntry.setCompressedSize(long)` method, then the compressed size will be set to the actual compressed size after deflation.

The current time will be used if the entry has no set modification time.

API Note:

When writing a directory entry, the STORED compression method should be used and the size and CRC-32 values should be set to 0:

```
ZipEntry e = new ZipEntry(entryName);
if (e.isDirectory()) {
    e.setMethod(ZipEntry.STORED);
    e.setSize(0);
    e.setCrc(0);
```

```
    }
    stream.putNextEntry(e);
```

This allows optimal performance when processing directory entries.

Parameters:

`e` - the ZIP entry to be written

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

closeEntry

```
public void closeEntry()
                throws IOException
```

Closes the current ZIP entry and positions the stream for writing the next entry.

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

write

```
public void write(byte[] b,
                  int off,
                  int len)
           throws IOException
```

Writes an array of bytes to the current ZIP entry data. This method will block until all the bytes are written.

Overrides:

`write` in class `DeflaterOutputStream`

Parameters:

`b` - the data to be written

`off` - the start offset in the data

`len` - the number of bytes that are written

Throws:

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O error has occurred

See Also:

`FilterOutputStream.write(int)`

finish

```
public void finish()
            throws IOException
```

Finishes writing the contents of the ZIP output stream without closing the underlying stream. Use this method when applying multiple filters in succession to the same output stream.

A ZipException will be thrown if the combined length, after encoding, of the entry name, the extra field data, the entry comment and CEN Header size, exceeds 65,535 bytes.

Overrides:

`finish` in class `DeflaterOutputStream`

Throws:

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O exception has occurred

close

```
public void close()
            throws IOException
```

Closes the underlying stream and the stream being filtered after the contents of the ZIP output stream are fully written.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `DeflaterOutputStream`

Throws:

`ZipException` - if a ZIP file error has occurred

`IOException` - if an I/O error has occurred

See Also:

`FilterOutputStream.flush()`, `FilterOutputStream.out`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.